

Ishara: Real-Time Sign Language Translator

Technical Setup Guide & Working Paper

SUMMARY

Ishara is a straightforward, open-source tool that reads Indian Sign Language (ISL) gestures through a basic webcam and outputs text and speech in real time. Instead of relying on expensive hardware or heavy graphics cards, it extracts 21 key points on the hand with MediaPipe, normalizes them relative to the wrist, and runs them through a light Random Forest model. It hits roughly 32 FPS on a basic dual-core laptop while keeping prediction accuracy around 96%.

Part 1: Setup & Technical Guide

1. Why I Built Ishara

Most hearing people don't know Indian Sign Language, which makes simple daily interactions frustrating for deaf individuals. When looking at existing projects, most required expensive sensory gloves or tried to pass full high-res camera frames into giant neural networks that crawl at 3 frames per second unless you have a dedicated gaming GPU.

Ishara was written to solve that: keep the processing light, run on any standard Python installation, and work on regular laptops with integrated graphics.

2. How the Pipeline Works

Every camera frame goes through four quick steps:

Webcam Input → Extract 21 Hand Points → Center & Scale Coordinates → Random Forest Guess
→ On-Screen Text + Audio

- Hand Joint Tracking:** MediaPipe detects 21 key locations on each hand (wrist, knuckles, finger joints, and tips). This drops the input from millions of camera pixels down to just 63 raw numbers (x, y, z for 21 points).
- Coordinate Normalization:** Raw pixel values change completely if you move your hand to the edge of the screen or lean back. To make the model position- and distance-independent:
 - *Centering:* Subtract the wrist's position (p_0) from every point so the wrist acts as $(0, 0, 0)$.
 - *Scaling:* Divide all offset points by the widest distance across your hand so big and small hands look the same to the model.
- Gesture Classification:** A Random Forest classifier (100 trees) reads the 63 normalized numbers and matches them against learned hand shapes in under 3 milliseconds.
- Text and Voice Output:** OpenCV draws the predicted letter or word right above your hand, while `pyttsx3` speaks it out loud.

3. Project Structure

```
Ishara/
├── data/
│   ├── raw_images/      # Saved image clips for checking
│   └── landmarks.csv    # Combined dataset of normalized hand coordinates
├── models/
│   └── ishara_model.p    # Saved model weights file
├── utils/
│   ├── dataset_builder.py # Helper tool to record your own signs
│   └── preprocess.py     # Code that converts video frames into 63 numbers
├── app.py               # Main live translation script
├── train.py             # Script to train and evaluate the classifier
├── requirements.txt     # Required Python libraries
└── README.md            # Project overview
```

4. Running Ishara on Your Machine

What You Need

- Python 3.8+
- A built-in or external USB camera

Installation

1. Get the code:

```
git clone https://github.com/ASMITSARKAR/Ishara.git
cd Ishara
```

2. Set up a virtual environment:

```
# On Linux or macOS:
python3 -m venv venv
source venv/bin/activate

# On Windows:
python -m venv venv
venv\Scripts ctivate
```

3. Install libraries:

```
pip install -r requirements.txt
```

4. Start live recognition:

```
python app.py
```

Press **Q** on your keyboard to close the video window. Press **M** to mute or unmute audio speech.

5. Adding Your Own Gestures

If you want to train Ishara on custom signs or your specific hand shape:

1. Collect training samples:

```
python utils/dataset_builder.py --label "Hello" --samples 200
```

2. Press **S** when ready. Hold your sign in front of the lens and tilt your hand slightly so the dataset captures slight variation.
3. Retrain the model:

```
python train.py
```

This reads the updated `data/landmarks.csv` file and saves a fresh model to `models/ishara_model.p`.

6. Practical Troubleshooting Tips

- **Hands aren't being picked up:** MediaPipe needs decent light. Avoid sitting directly in front of a bright window where your hand turns into a dark silhouette.
- **Model confuses similar signs (like 'M' and 'N'):** Since both signs look similar from the front, record extra training images with your hand tilted at slight angles.

Part 2: Methodology & Results Paper

1. Abstract

Indian Sign Language (ISL) is widely used across South Asia, yet very few non-signers understand it. This paper presents Ishara, a vision-based tool for live ISL recognition. By separating spatial feature extraction from image processing, the system reduces raw camera frames down to 63 normalized keypoint values. Across test samples of static ISL alphabets and common phrases, Ishara scored 96.4% classification accuracy while maintaining 32.4 FPS on standard consumer CPUs.

2. Background & Motivation

Sign language translation systems usually fall into two categories: sensor gloves or deep learning image models. Sensor gloves are accurate but uncomfortable and expensive. Image-based convolutional networks work well but require continuous GPU compute, making them hard to deploy on ordinary laptops or low-power single-board computers.

Ishara bridges this gap. By converting video frames into localized landmark vectors before classification, the system discards background clutter, skin tone variances, and lighting shifts while dramatically reducing computational overhead.

3. Mathematical Method

1. Extracting Keypoints

Video frames are captured at 640×480 pixels. MediaPipe Tracks 21 3D hand joints $p_i = (x_p \ y_p \ z_p)$ for $i = 0, 1, \dots, 20$, where p_0 is the center of the wrist.

2. Spatial Transformations

To ensure signs are recognized regardless of where the hand is on screen or how close it is to the camera, coordinates undergo two basic transformations:

1. Origin Offset: $p'_i = (x_i - x_0 \ y_i - y_0 \ z_i - z_0)$

2. Bounding Distance Scale: $S = \max_i \sqrt{(x'_p)^2 + (y'_p)^2 + (z'_p)^2}$

Normalized Feature Coordinate: $\hat{p}_i = p'_i / S$

This outputs a simple 63-element feature vector V :

$$V = [\hat{x}_0 \ \hat{y}_0 \ \hat{z}_0 \ \hat{x}_1 \ \hat{y}_1 \ \hat{z}_1 \ \dots \ \hat{x}_{20} \ \hat{y}_{20} \ \hat{z}_{20}]$$

3. Model Choice

We train a Random Forest model with 100 decision trees. Because decision tree evaluation involves simple conditional splits on a tiny 63-element array, inference takes less than 3 milliseconds per frame.

4. Test Results

We evaluated the system on static ISL alphabets and high-frequency phrase signs, taking 200 frame captures per gesture under typical office lighting. The data was split 80% for training and 20% for testing.

Test Metric	Result
Overall Test Accuracy	96.4%
Macro Precision	0.962
Macro Recall	0.965
Macro F1-Score	0.963
Hand Detection Latency	12.1 ms per frame
Model Prediction Time	2.8 ms per frame
Total Processing Speed	32.4 FPS (Tested on Intel i5 CPU)

5. Summary & Future Scope

Ishara demonstrates that real-time sign language translation doesn't require heavy deep learning networks or custom hardware. Combining skeletal landmark extraction with coordinate normalization creates a fast, reliable, and accessible translation tool for everyday devices. Future plans include testing lightweight RNN/GRU models for continuous multi-word sentence decoding.

References

1. C. Lugaresi et al., "MediaPipe: A Framework for Building Perception Pipelines," *arXiv preprint arXiv:1906.08172*, 2019.
2. P. Garg, X. Aggarwal, and S. Sofat, "Vision Based Hand Gesture Recognition for Indian Sign Language: A Review," *International Journal of Computer Applications*, vol. 84, no. 16, pp. 38–43, 2013.
3. L. Breiman, "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.